```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

image_path = "/content/Golden Retriever.png"
img = cv2.imread(image_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 6))
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```



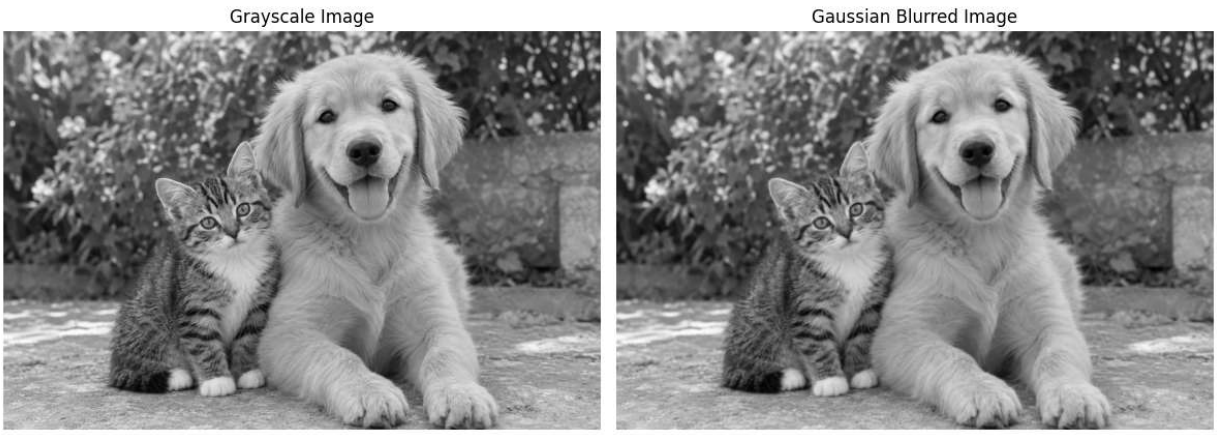

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(blurred, cmap='gray')
plt.title("Gaussian Blurred Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
_, global_thresh = cv2.threshold(
    blurred, 127, 255, cv2.THRESH_BINARY
)

plt.figure(figsize=(8, 6))
plt.imshow(global_thresh, cmap='gray')
plt.title("Global Thresholding")
plt.axis("off")
plt.show()
```




```
otsu_value, otsu_thresh = cv2.threshold(
    blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Otsu Threshold Value:", otsu_value)

plt.figure(figsize=(8, 6))
plt.imshow(otsu_thresh, cmap='gray')
plt.title("Otsu's Thresholding")
plt.axis("off")
plt.show()
```

```
Otsu Threshold Value: 122.0
```




```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(global_thresh, cmap='gray')
plt.title("Global Thresholding")
plt.axis("off")

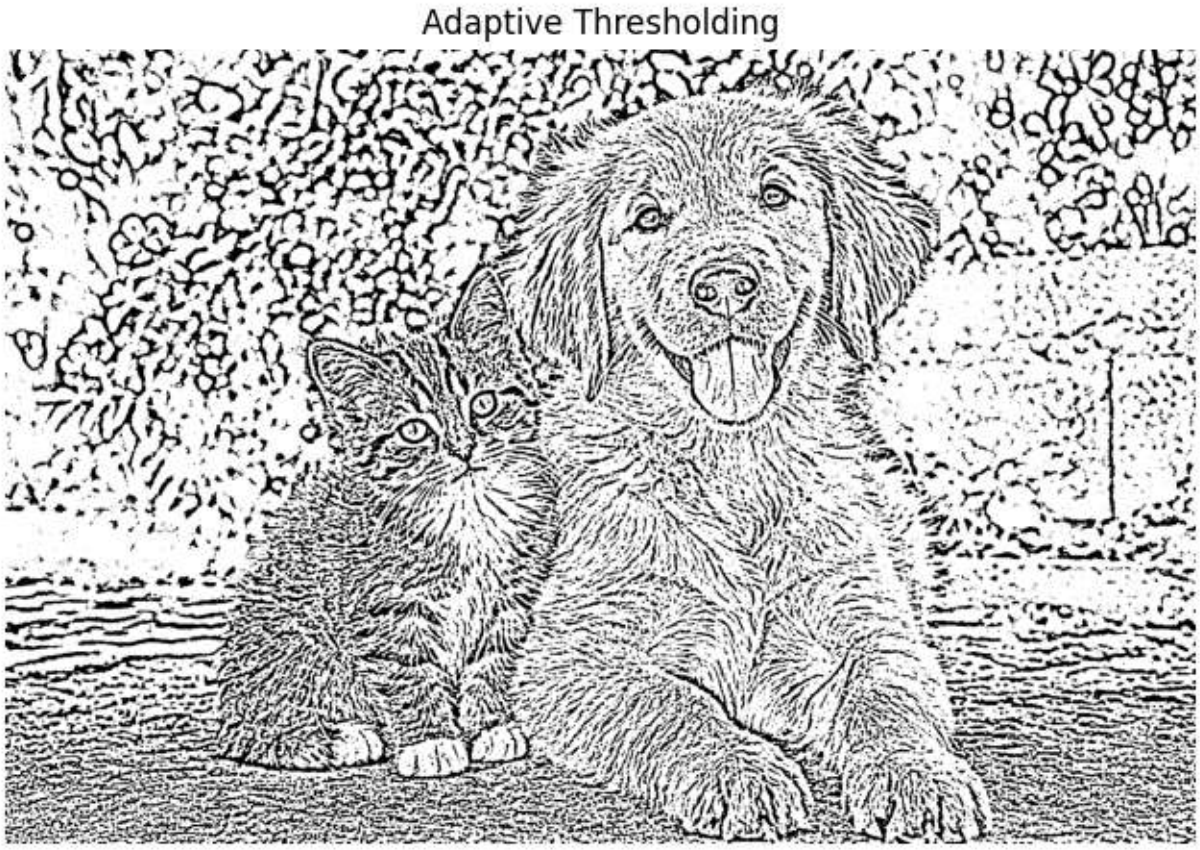
plt.subplot(1, 2, 2)
plt.imshow(otsu_thresh, cmap='gray')
plt.title("Otsu's Thresholding")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
adaptive_thresh = cv2.adaptiveThreshold(
    blurred,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

plt.figure(figsize=(8, 6))
plt.imshow(adaptive_thresh, cmap='gray')
plt.title("Adaptive Thresholding")
plt.axis("off")
plt.show()
```




```
_, binary = cv2.threshold(
    blurred, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

plt.figure(figsize=(8, 6))
plt.imshow(binary, cmap='gray')
plt.title("Binary Image for Watershed")
plt.axis("off")
plt.show()
```




```
kernel = np.ones((3, 3), np.uint8)
sure_bg = cv2.dilate(binary, kernel, iterations=3)
dist_transform = cv2.distanceTransform(
    binary, cv2.DIST_L2, 5
)

_, sure_fg = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(sure_fg, cmap='gray')
plt.title("Sure Foreground")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(unknown, cmap='gray')
plt.title("Unknown Region")
```

```
plt.axis("off")

plt.tight_layout()
plt.show()
```

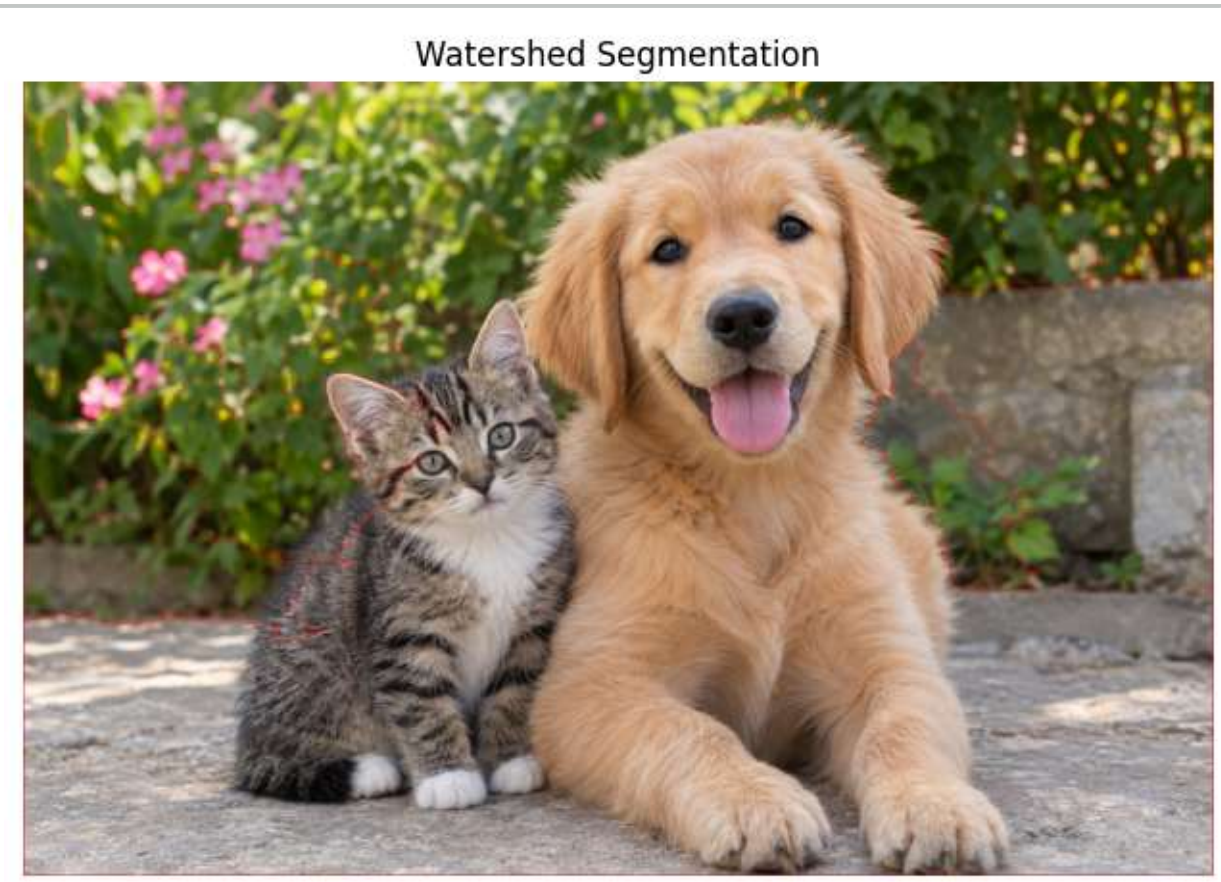



```
num_labels, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

watershed_img = img.copy()
markers = cv2.watershed(watershed_img, markers)

watershed_result = img_rgb.copy()
watershed_result[markers == -1] = [255, 0, 0]

plt.figure(figsize=(8, 6))
plt.imshow(watershed_result)
plt.title("Watershed Segmentation")
plt.axis("off")
plt.show()
```




```
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
pixel_data = rgb.reshape((-1, 3))
pixel_data = np.float32(pixel_data)
k = 3

kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixel_data)
centers = np.uint8(kmeans.cluster_centers_)
segmented_data = centers[labels]
segmented_image = segmented_data.reshape(rgb.shape)

plt.figure(figsize=(8, 6))
plt.imshow(segmented_image)
plt.title("K-Means Segmentation (K=3)")
plt.axis("off")
plt.show()
```




```
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_thresh, cmap='gray')
plt.title("Global Thresholding")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap='gray')
plt.title("Otsu's Thresholding")
plt.axis("off")

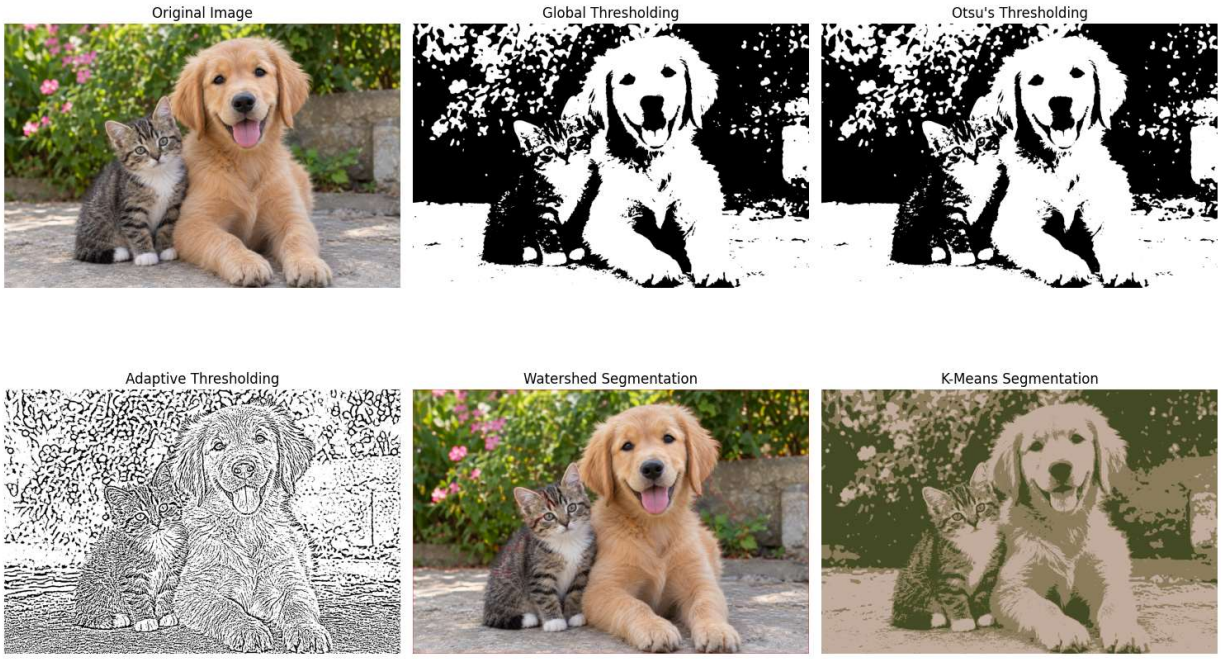
plt.subplot(2, 3, 4)
plt.imshow(adaptive_thresh, cmap='gray')
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(watershed_result)
plt.title("Watershed Segmentation")
```

```
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(segmented_image)
plt.title("K-Means Segmentation")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Start coding or generate with AI.

```
Start coding or generate with AI.
```

Observations

Global Thresholding: Separates pixels using a fixed threshold but may not handle varying illumination well.
Otsu's Thresholding: Automatically selects a threshold and generally provides better separation than a manually selected threshold.
Adaptive Thresholding: Uses local thresholds and performs better when illumination varies across the image.
Watershed: Identifies boundaries and can help separate connected or touching regions.
K-Means: Groups pixels based on color and is suitable for segmenting colorful images.
Overall Observation

For this image
K-Means is particularly useful for color-based segmentation.